MusX User and Developer Manual

A browser-based node patcher for sound and music

Paul Fishwick and Claude Code

July 2026

Contents

1 Introduction

MusX is a single-page, browser-based *node patcher* for building sounds and music. A patch is a graph of typed objects ("nodes") wired together by cables; the graph is evaluated by the Web Audio API [3] through the Tone.js library [4]. MusX evolved from studying two seminal visual dataflow environments for computer music: Miller Puckette's Pure Data (Pd) [1] and Cycling '74's Max/MSP [2]. From those systems MusX borrows the central idea that a musician builds an instrument by *wiring objects together* rather than writing a program in text, along with specific conventions such as the tilde (~) suffix on signal-rate objects, inlets on the top of a box and outlets on the bottom, and the distinction between audio signals and control messages. MusX deliberately keeps a much smaller object set than Pd or Max, aiming for a simple but comprehensive core that runs with no installation in any modern browser. More recently MusX has grown a *sound-transformation* side inspired by the Composers Desktop Project (CDP) [6] and its node-based front end SoundThread [7]. Where CDP transforms recorded sound offline through command-line tools, MusX reimplements the same ideas as *real-time* Web Audio objects: soundfile and microphone inputs (Section 5) feeding granular and other transformations that process live.

As shown in Figure 1, the workspace is a dot-grid canvas populated with object boxes. Audio cables are drawn thick and orange; control cables are thin and green. A toolbar across the top hosts the global controls (audio on/off, transport, tempo, master level, and patch file management).

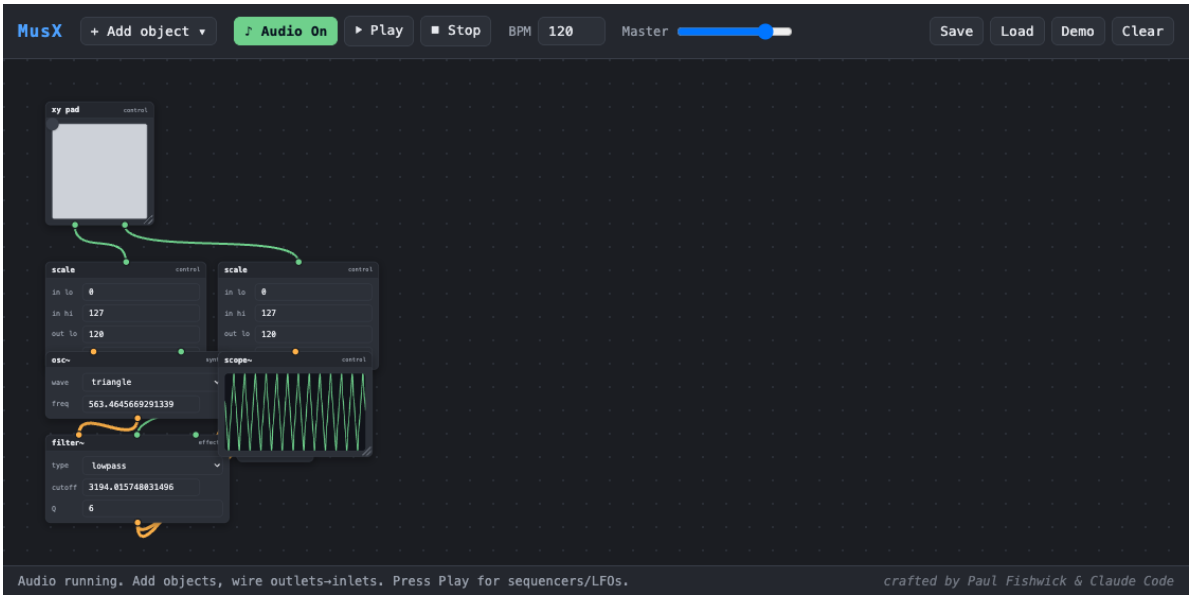

Figure 1: The MusX workspace with the "XY Synth" patch loaded — an `xy pad` drives a `tri~` oscillator through two `scale` objects and a resonant `filter~` into the output, with a `scope~` showing the waveform. This patch reproduces the subtractive-synth example from the Max/MSP product page [2].

1.1 Design Goals

- **No build step, no install.** MusX is plain HTML, CSS, and ES-module JavaScript. Tone.js is vendored locally; nothing is fetched at runtime except the optional Python runtime (Section 7).

- **Local-first.** Everything runs in the browser on the user's machine.
- **Signal vs. message.** As in Pd and Max, MusX separates audio signals (continuous, sample-rate) from control messages (discrete events).
- **One object = one definition.** Adding a new object is a single registry entry; the graph engine never special-cases an object type (Section 12).
- **Resizable, legible UI.** Every graphical object can be resized, and any object carrying a frequency shows the corresponding musical note.

2 Installation and Setup

MusX has been developed and tested on macOS with a Chromium-based browser. Because it uses ES modules, it must be served over HTTP rather than opened from the filesystem. The repository root contains two shell scripts:

```
./start # serves http://localhost:8123 (prints the URL)
./stop # stops the server (robust: stops by port even if the pid file is stale)
```

`./start` first frees its port: if any process (a stale server or another application) is already listening there, it is closed before MusX takes the port. After `./start`, open the printed `http://localhost:8123/index.html`. Click **Start Audio** once (browsers require a user gesture before audio may begin), then build a patch or load a demo.

3 Interface Tour

3.1 Canvas and Objects

Object boxes are dragged by their title bar. Each box has *inlets* along the top and *outlets* along the bottom, drawn as small circles colour-coded by type: orange for audio, green for control. Selecting a box (single click) and pressing `Delete` removes it.

3.2 Launch Default and Info Boxes

On launch MusX loads the **Cathedral Pad** patch (Section 9) as a ready-to-play default; a dismissable hint in the bottom-right corner notes that it can be reloaded from **Demo → 11 Cathedral Pad**. A second box may appear in the top-right corner: patches that carry an optional `credits` field (`text` + `url`) — such as the MIDI demos — show that credit and a source link there.

3.3 Adding Objects

Objects are added from a two-level menu organised by function, shown in Figure 2. The menu opens either from the **+ Add object** button in the toolbar or by right-clicking empty canvas; hovering a category reveals its objects in a submenu. Right-click drops the new object where you clicked; the button drops it at a staggered default position.

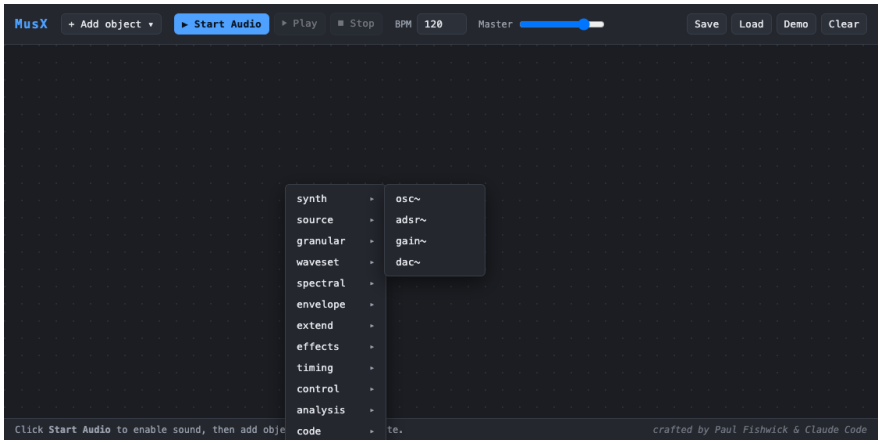

Figure 2: The two-level "Add object" menu. Top level lists the functional categories; hovering a category (here `synth`) reveals its objects to the right.

3.4 Navigating: Pan, Zoom, and Fit

Patches can grow larger than the visible canvas. Scroll the mouse wheel to **zoom** in and out (the zoom centres on the cursor); drag an empty part of the canvas (or use the middle mouse button) to **pan**. Double-clicking empty canvas **fits** all objects into view. Loading a patch automatically fits it, so an incoming patch is always framed completely regardless of its size. Object dragging and object placement remain accurate at any zoom level.

3.5 Wiring and Deleting Cables

Drag from an outlet to an inlet to create a cable. Audio outlets connect only to audio inlets and control to control; mismatches are rejected. A cable highlights red on hover; clicking it deletes it.

3.6 Resizing Graphical Objects

Every graphical object — `xy pad`, `scope~`, `plot`, `keyboard`, `step seq`, `slider`, `bang`, `message`, and `code` — has a resize grip in its bottom-right corner. Dragging it rescales the object (the keyboard rescales its keys, the grid its cells, canvases their drawing area). The size is stored in the patch and restored on load.

3.7 Frequency and Note Display

Any object whose parameters include a frequency shows the nearest musical note and octave immediately after the number, as in Figure 3. The note is shown only when the frequency lies within a few cents of an equal-tempered pitch (for example 440 Hz → A4, 1760 Hz → A6); frequencies that are not notes leave the label blank. The on-screen keyboard labels each C with its octave (C3, C4, . . .).

3.8 Audio Lifecycle: Start Audio, Play, Stop

Start Audio unlocks the browser audio context and builds the live signal graph; continuously sounding objects (free-running oscillators) begin immediately. **Play** runs the global transport so that time-based objects (`metro`, `step seq`, the `funcgen` control sweep) fire, and ramps the master

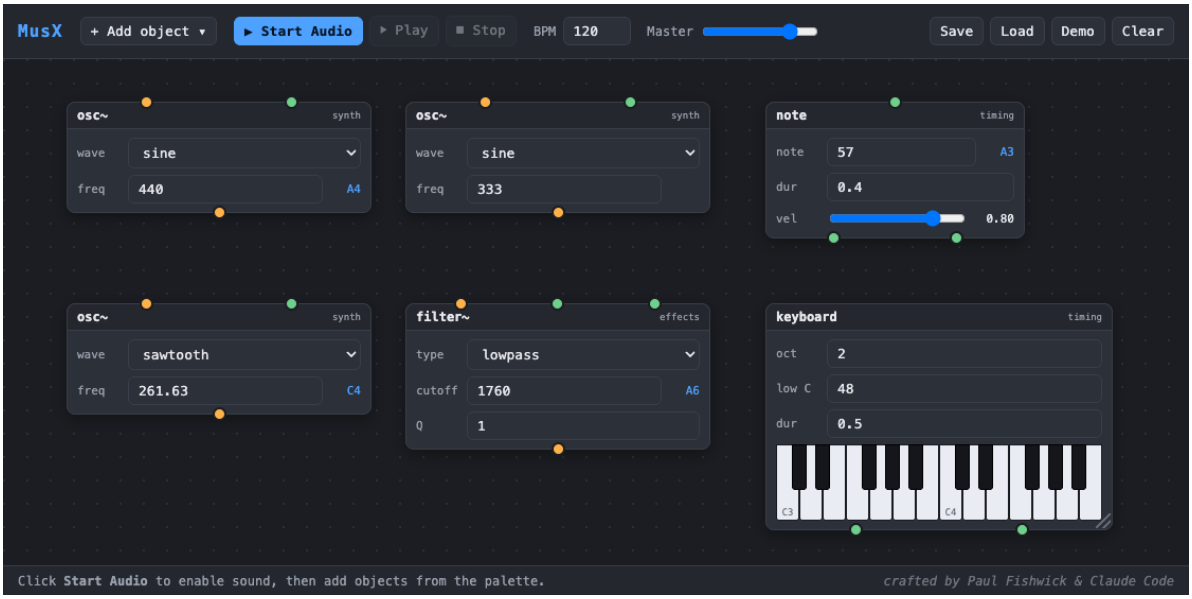

Figure 3: Frequency-to-note display. The `osc~` at 440 Hz shows A4 and at 261.63 Hz shows C4; 333 Hz shows no note; the `filter~` cutoff of 1760 Hz shows A6; the `note` object shows A3. The keyboard's C keys are labelled with their octaves.

gain up. **Stop** pauses the transport and ramps the master gain to zero. While stopped, `scope~` and `plot` draw a flat line so the visuals match the silence.

3.9 Saving and Loading

Save downloads the current patch as JSON (object types, positions, parameter values, widget sizes, and all cables); **Load** restores one. Seven example patches ship in `patches/` and are also available from the **Demo** button.

4 Object Catalog

Objects are grouped by function. Audio inlets/outlets are marked with a tilde in the interface. Table 1 summarises the current set.

Table 1: The MusX object catalog.

Category	Object	Function
source	`sndfile~`	Soundfile player: load/drag a WAV or pick a bundled sound; loop, varispeed, `trig`, and a Start-Mod random start offset (Section 9)
	`sampler~`	Play a sample chromatically (varispeed): `freq`→playback rate about a `root` pitch, with a hold-to-sustain gate (Section 9)
	`mic~`	Live microphone input; gain makeup and a live input-level (dB) meter
synth	`osc~`	Oscillator (sine/square/saw/triangle); freq control + audio-rate FM inlet

Category	Object	Function
	`unison~`	Fat unison oscillator: up to 8 detuned, stereo-spread voices in one box (Section 9)
	`pan~`	Equal-power stereo panner (position $-1 \ldots 1$)
	`spat~`	3D binaural panner (HRTF): $x/y/z$ place a source around the listener
	`adsr~`	Attack–decay–sustain–release envelope (gated note-on/off); optional velocity→amp
	`gain~`	Amplitude / level
	`dac~`	Audio output (to speakers via the master)
effects	`filter~`	Lowpass/highpass/bandpass/notch with cutoff + Q
	`delay~`	Feedback delay (time, feedback, wet)
	`reverb~`	Reverb (decay, pre-delay in ms, wet)
	`dist~`	Distortion (amount, wet)
granular	`grain~`	Granular cloud of a buffer (grain size, overlap, time, pitch, reverse)
	`tstretch~`	Time-stretch: change speed/duration, keep pitch
	`pshift~`	Pitch-shift: change pitch, keep speed/duration
waveset	`wsdistort~`	Waveset distortion: repeat/omit/reverse/average/telescope/reform of zero-crossing wavesets
spectral	`spec.freeze~`	Phase-vocoder spectral freeze: hold the current frame (`trig` re-captures)
	`spec.blur~`	Average FFT magnitudes over N frames (spectral smear)
	`spec.filter~`	Spectral gate: keep bins above (or below) a fraction of peak level
	`spec.pitch~`	Phase-vocoder transpose (pitch in semitones; duration unchanged)
	`spec.stretch~`	Spectral partial-stretch (frequency-axis; harmonic → inharmonic)
	`spec.morph~`	Interpolate the spectra of two inputs (timbre crossfade)
envelope	`envfollow~`	Amplitude-envelope follower (audio-rate `env` + control `val`)
	`envimpose~`	VCA: multiply a carrier by an incoming envelope
extend	`iterate~`	Triggered stutter: repeat the last segment with decay + pitch step
	`scramble~`	Chop into segments and replay them shuffled / drunk
timing	`transport`	Sets global tempo (BPM)
	`metro`	Emits a bang every musical interval
	`bang`	Manual trigger button
	`note`	Emits a note (frequency + duration) from a MIDI number
	`keyboard`	On-screen piano; emits note + frequency
	`step seq`	Piano-roll step sequencer
	`midifile`	Plays a Standard MIDI File; poly voice allocator or mono/legato; track isolate, skip, transpose, loop

Category	Object	Function
control	`number`, `slider`	Numeric value sources
	`math`, `scale`	Arithmetic; linear range mapping (`scale`)
	`chord`	Root frequency → chord tones (quality + number of notes; Section 9)
	`message`	Emits a stored value on click
	`xy pad`	2-D control surface (X and Y, 0–127)
	`breakpoint~`	Draw-and-play automation curve (control stream)
	`scope~`	Oscilloscope
analysis	`plot`	Rolling XY graph of a control stream
	`funcgen`	Algebraic function generator (Section 6)
code	`code`	JS/Python control-rate code object (Section 7)
patch	`patcher`	A sub-patch as one box; edit its inner graph inside (Section 8)
	`inlet~`/`inlet`	Audio/control input boundary of a patcher
	`outlet~`/`outlet`	Audio/control output boundary of a patcher

5 Sound Sources and CDP-Style Processing

In addition to synthesis, MusX can transform *recorded* sound in real time, in the spirit of the Composers Desktop Project (CDP) [6] and its node-based front end SoundThread [7]. Where CDP runs offline command-line tools that read and write soundfiles, MusX reimplements the same ideas as live Web Audio objects.

5.1 Soundfile and Microphone Inputs

The `sndfile~` object plays an audio buffer: load a WAV with its *file* button or by dragging one onto the box, or choose a bundled sound from its grouped dropdown. It loops, plays at a variable *rate* (varispeed), and restarts on a `trig`. The `mic~` object brings in live microphone audio; because laptop microphones are quiet it provides makeup gain (0–8) and a live input-level meter in decibels, so one can see the signal arriving. Both objects are shown in Figure 4. **Use headphones** with a live microphone: through the speakers the microphone hears the output and howls.

5.2 Granular Transformation

The granular objects granulate a buffer using overlapping short grains, which decouples time from pitch. `grain~` is the full cloud (grain size, overlap, scan speed, transposition, reverse); `tstretch~` changes speed while preserving pitch; and `pshift~` transposes while preserving speed. Figure 5 shows two granular voices feeding a shared reverb. Because a granular object reads its own buffer (rather than a live stream), it loads a file exactly as `sndfile~` does.

5.3 Waveset Distortion

A *waveset* is the fragment of a signal between alternate zero-crossings—from one upward zero-crossing to the next. For a pure tone this is one cycle; for complex sound it is a "pseudo-cycle". CDP's DISTORT family rebuilds a sound from transformed wavesets, and `wsdistort~` reimplements it in real time on the live stream. A single *mode* menu selects the transformation:

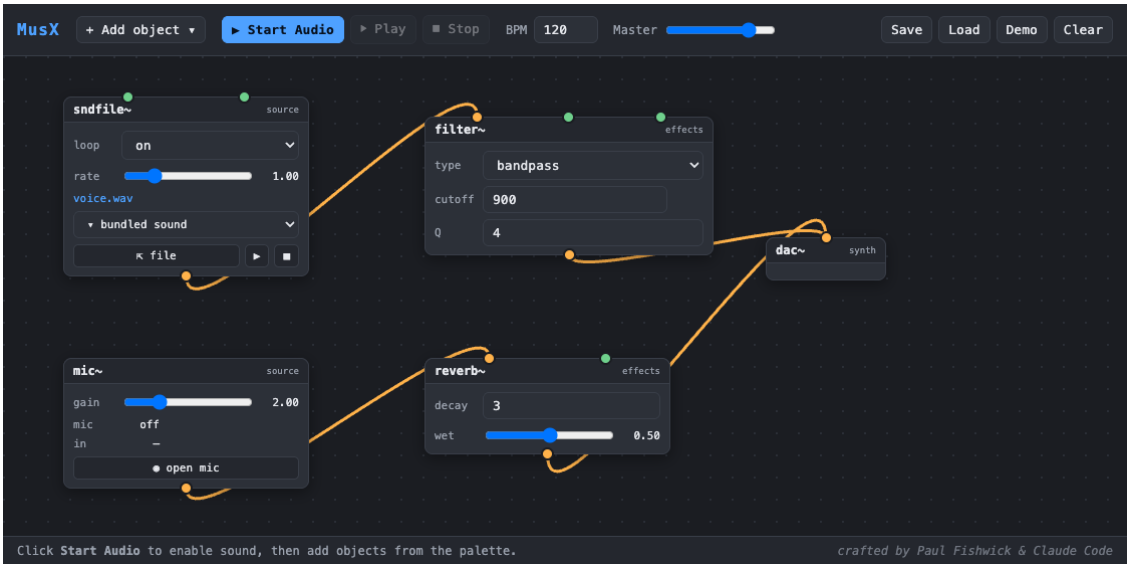

Figure 4: The two sound-input objects. `sndfile~` (with a bundled sound loaded) feeds a bandpass `filter~`; `mic~` feeds a `reverb~`; both reach the output.

- **repeat** — emit each waveset several times (*count*); time-extends the sound and adds a sub-octave buzz.
- **omit** — keep some wavesets and drop others (*keep*/*skip*) for rhythmic gating and thinning.
- **reverse** — reverse each waveset in place, adding grit at constant length.
- **average** — replace a group of wavesets with their averaged shape, smearing timbre and pitch.
- **telescope** — squeeze a *group* of wavesets into one, contracting time and raising pitch.
- **reform** — substitute a simple waveform (*shape*) scaled to each waveset's length and peak, swapping timbre while keeping the rhythm.

The *group* parameter sets how many consecutive wavesets an operation treats as a unit (CDP's `cyclecnt`), and *level* is an output gain. Because `repeat`, `omit`, and `telescope` change the number of samples, the object buffers its output internally; the buffer is bounded (about one second) so latency cannot grow without limit—the one concession that a live implementation makes to CDP's offline processing. Figure 6 shows the object crunching a sustained drone. The distortion is 100 % wet, matching CDP; patch a parallel path if a dry blend is wanted.

5.4 Spectral Transformation (Phase Vocoder)

The spectral objects work in the frequency domain. The browser's analysis node can read a spectrum but cannot resynthesize one, so MusX includes a real *phase vocoder* built as a custom audio worklet: a sliding Short-Time Fourier Transform (2048-point FFT, 75 % overlap, Hann window) analyses the signal into magnitude and phase per frequency bin, an operation modifies those bins, and an inverse transform with overlap-add reconstructs a continuous stream. Because the transform window is finite, the objects add about 35 ms of latency. All five are audio-in → audio-out and leave the sound's *duration* unchanged.

Figure 5: The "Granular Cloud" patch (`patches/granular/`): `grain~` scans a drone slowly while `pshift~` transposes a vowel up a fifth; both are mixed into a reverb, with a `scope~` on the output.

- `spec.freeze~` — captures one spectral frame and holds it indefinitely, advancing each bin's phase at its analysed frequency so the freeze sustains smoothly rather than buzzing. Toggle *freeze*; send a `trig` to re-capture the current sound.
- `spec.blur~` — averages each bin's magnitude over the last N frames (*frames*), smearing transients and movement into a sustained wash.
- `spec.filter~` — a spectral gate that keeps only bins louder than a fraction of the frame's peak (*thresh*); *invert* keeps the quiet bins instead, for denoising or for isolating a noise bed.
- `spec.pitch~` — transposes by estimating each bin's true frequency from its phase advance and moving it to a new bin, so pitch changes (*semis*) while speed and duration stay fixed. This is genuine pitch-shifting, distinct from varispeed.
- `spec.stretch~` — stretches the spacing of the partials along the frequency axis (*stretch*): above 1 the overtones spread apart, turning a harmonic tone inharmonic and bell-like; below 1 they compress. This is a *spectral* stretch, not a time-stretch (for time-stretch use the granular `tstretch~`), and so it too leaves duration unchanged.
- `spec.morph~` — takes *two* audio inputs (*a* and *b*) and interpolates their spectra — both magnitudes and per-bin frequencies — by *morph* (0 = a, 1 = b). Unlike a gain crossfade, which simply plays two sounds at once in the middle, this builds a genuine hybrid spectrum, so a voice can turn gradually into a bell.

Figure 7 shows the partial-stretch object turning a drone into a bell. All of these parameters are modulatable, so an LFO or envelope can sweep the blur window, freeze threshold, transposition, or stretch amount over time.

5.5 Extend, Modify, and Envelope

A final family reshapes sound in the time domain and works with amplitude envelopes.

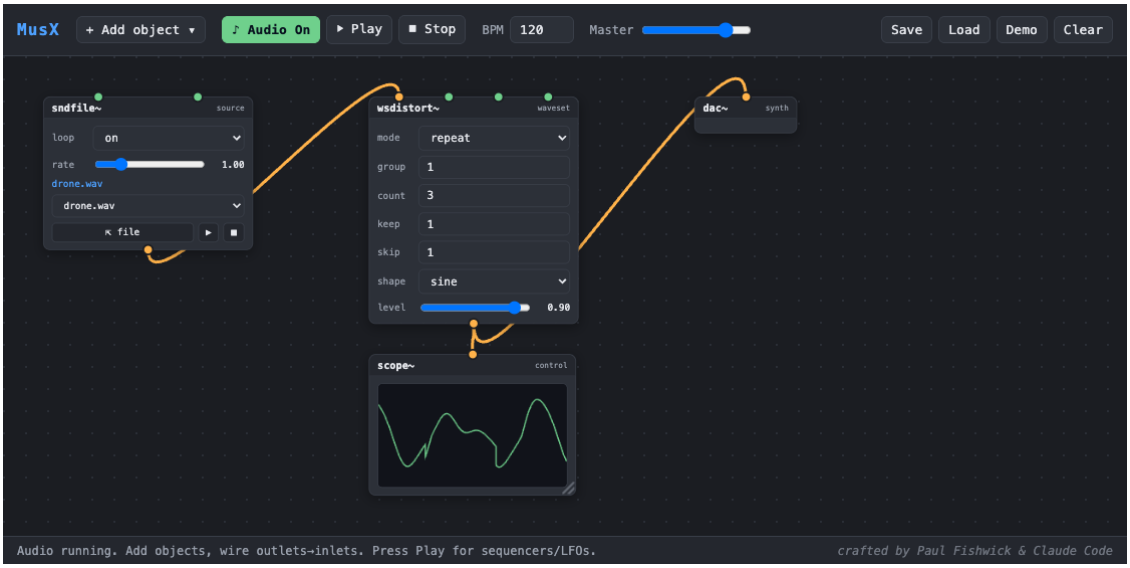

Figure 6: The "Waveset Crunch" patch (`patches/cdp/`): `sndfile~` auto-loads a drone into `wsdistort~`, whose transformed output reaches the speakers and a `scope~`. The *mode* menu selects the waveset operation.

Envelope objects. `envfollow~` tracks the amplitude envelope of its input and offers it two ways: as an audio-rate signal on `env` (to drive another object's level) and as a 0–1 control stream on `val` (to plot or to modulate a parameter). `envimpose~` is a voltage-controlled amplifier: it multiplies a carrier by an incoming envelope. Patching `envfollow~` on a drum loop into `envimpose~` on a sustained pad makes the pad speak the rhythm of the drums, as in Figure 8.

Breakpoint automation. `breakpoint~` provides the real-time equivalent of a CDP breakpoint file: a small canvas holding a line-segment curve that is edited by hand — click to add a point, drag to move it, right-click to delete — with the two endpoints pinned in time. When the transport runs, a playhead sweeps the curve over *dur* seconds (looping or one-shot) and emits the interpolated value, scaled to *lo*–*hi*, on its `val` outlet. Wired into any modulatable parameter it becomes a drawable automation lane; the curve is saved with the patch.

Extend / glitch objects. `iterate~` continuously records its input and, when it receives a `trig`, grabs the last *seg* milliseconds and repeats it *count* times, each repeat quieter (*decay*) and transposed by a *pitch* step — a triggered stutter or echo burst. `scramble~` chops the input into *seg*-length segments, keeps the most recent few, and replays them in shuffled or drunk-walk order; it emits one segment for every segment it records, so it stays locked to real time rather than drifting.

5.6 Modulatable Parameters

Any parameter may be declared *modulatable*, in which case it exposes an optional control inlet named after the parameter. A cable from a `funcgen` LFO, a `slider`, or an `xy pad` then drives that parameter continuously, while its own widget still works when nothing is patched. For example, wiring a slow `funcgen` into the `rate` inlet of `sndfile~` automates the tape-style varispeed wobble that is expressive to perform by hand. Modulation drives the audio directly and does not move (or overwrite) the widget's stored value, so the effect is heard rather than seen. The granular objects expose their grain, overlap, rate, and pitch parameters this way — patching an LFO into `pshift~`'s pitch inlet, for instance, produces vibrato — and `wsdistort~` likewise exposes its *group*, *count*, and

Figure 7: The "Spectral Stretch" patch (`patches/cdp/`): `sndfile~` auto-loads a drone into `spec.stretch~`, whose inharmonic output passes through a `reverb~` to the speakers and a `scope~`.

level parameters, so a slow ramp into *count* sweeps the waveset repetition.

5.7 Sample Library and Grouped Patches

Sample sounds ship under `sounds/`, organised into sub-directories (`tonal/`, `vocal/`, `texture/`) whose names become the group headings in the soundfile dropdown. Example patches ship under `patches/`, likewise grouped into sub-directories (`cdp/`, `granular/`, `live/`). A patch may reference a bundled sound by its path so that the sound auto-loads when audio starts; saved patches store this path (not the audio itself), so bundled sounds reload automatically while a user's own imported files are re-selected on load.

6 The `funcgen` Expression Language

The `funcgen` object generates a function specified algebraically, in the spirit of Max's `expr~`. The expression is written in terms of a phase variable `t`. It is evaluated two ways simultaneously: the `sig` outlet plays one cycle of the expression over $\texttt{t} \in [0, 1)$ as an audible wavetable oscillator, and the `val` outlet samples the expression over time as a control stream suitable for feeding a `plot` (Figure 9). The parser is sandboxed — it never calls JavaScript `eval` on user input — and supports `+ - * / ^`, parentheses, the constants `pi`, `e`, `tau`, and the functions `sin cos tan asin acos atan exp log ln sqrt abs floor ceil round sign min max mod` plus oscillator shapes `saw square tri pulse`.

7 Code Objects

The `code` object runs user-written code at control rate: it reads its inlets `a` and `b`, runs the code whenever an input arrives, and emits the result. Two languages are supported. **JavaScript** runs natively (a bare expression such as `a*b+1` or a function body with `return`). **Python** runs through

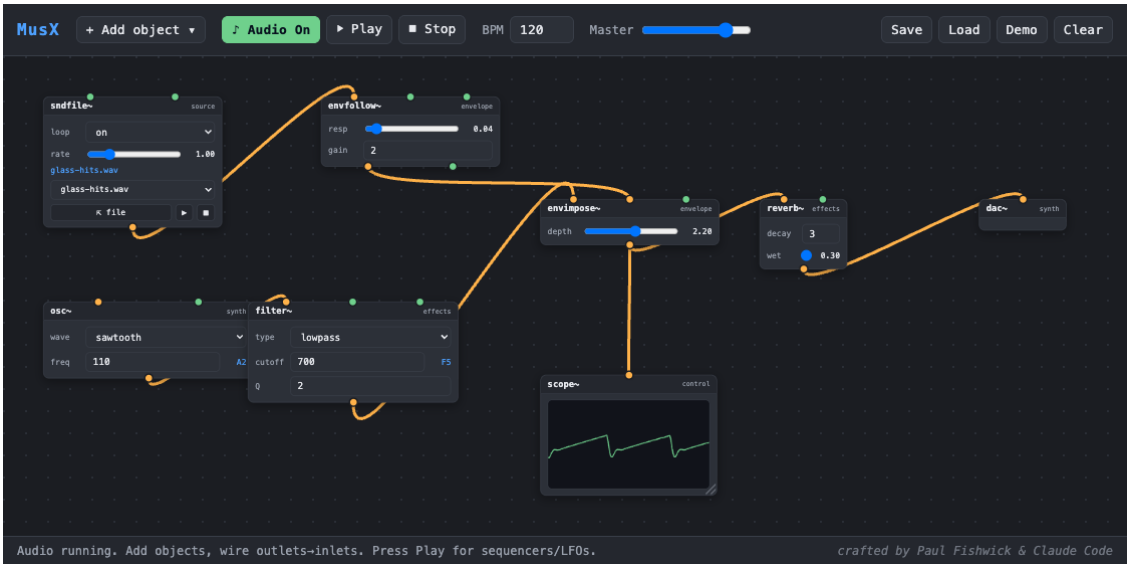

Figure 8: The "Envelope Impose" patch (`patches/cdp/`): a percussive `sndfile~` drives `envfollow~`, whose envelope opens the `envimpose~` VCA on a sustained saw pad, so the pad pulses in the rhythm of the source.

the Pyodide WebAssembly runtime [5], which is downloaded from its CDN the first time a Python code object executes; JavaScript code objects never trigger that download. Figure 10 shows a patch in which a `message` sends a MIDI number to a `code` object that converts it to hertz, while a `bang` button triggers the envelope.

Security note. Because a code object executes the code it contains, loading a patch file authored by someone else will run that code — exactly as opening a Max patch containing a `js` object would. This is safe for your own patches; treat untrusted patch files with the same caution as any downloaded program.

8 Subpatches and Abstractions

A `patcher` object is a box whose *inside* is another patch, so a group of objects can be reused as a single block — the MusX equivalent of a Max sub-patch or Pd abstraction. Double-clicking a `patcher` **descends** into its inner graph; a breadcrumb trail at the top-left of the canvas (`root patch ▷ patcher`) shows the current depth, and clicking a breadcrumb segment **climbs** back out. Nesting is arbitrary — patchers may contain patchers.

The box's ports are defined by *boundary objects* placed inside it: each `inlet~` or `inlet` adds one input port (audio or control) and each `outlet~` or `outlet` adds one output, ordered left-to-right by position. The engine runs a patcher as a nested runtime: audio and control cross the boundary transparently, so a subpatch behaves exactly like a hand-built chain (Figure 11).

Encapsulating a selection. Rather than building a box from scratch, several existing objects can be collapsed into one. Shift-click objects (or shift-drag a rubber-band box) to multi-select, then press `Cmd/Ctrl+E` or the **Encapsulate** toolbar button. MusX moves the selected objects into a

Figure 9: A `funcgen` evaluating `sin(2*pi*t) + 0.3*sin(6*pi*t)`. The `plot` (blue) shows the function sampled over time; the `scope~` (green) shows the same function rendered as an audible wavetable — note the matching harmonic ripple.

new `patcher`, automatically creating a boundary object for every cable that crossed the selection and rewiring those cables to the new box's ports.

File-referenced abstractions. A `patcher` can instead *reference* a saved `.json` patch file under the served tree, so one definition can be instanced in many places. **Insert abstraction** creates such a box from a path; **Save as abstraction** downloads the current subpatch as a reusable file; and **Reload abstractions** re-fetches every referenced file so an edit to the definition propagates to all instances at once. A referenced box is read-only when opened (a banner marks it so), with a **Detach** action to fork it into a private, editable inline copy.

9 Fat Synth Voices: Unison, Panning, and Chords

Rich, wide, "cinematic" synth textures come less from adding more oscillators than from a specific architecture: *unison voice stacking*, *detuning*, and *stereo spreading* (and, for sampled sources, a randomised start time). MusX packages this as a small family of objects, inspired by the layered-oscillator design of instruments such as Hexeract [8].

`unison~`. A single oscillator *module* that internally runs up to eight voices of the chosen waveform. Each voice is detuned by a symmetric offset in *cents* (so the perceived width is the same at every pitch) and placed at its own position across the stereo field by an equal-power panner; the voices are summed with a $1/\sqrt{N}$ scaling to keep loudness roughly constant as the count changes. Its parameters are `wave`, `voices` (1–8), `detune` (cents), `spread` (0–1), and `level`; a single `freq` control fans out to all voices, and changing the voice count rebuilds the stack live. One `unison~` already sounds like a small ensemble; stacking a few (with different waveforms and detune amounts) yields the classic "supersaw" pad.

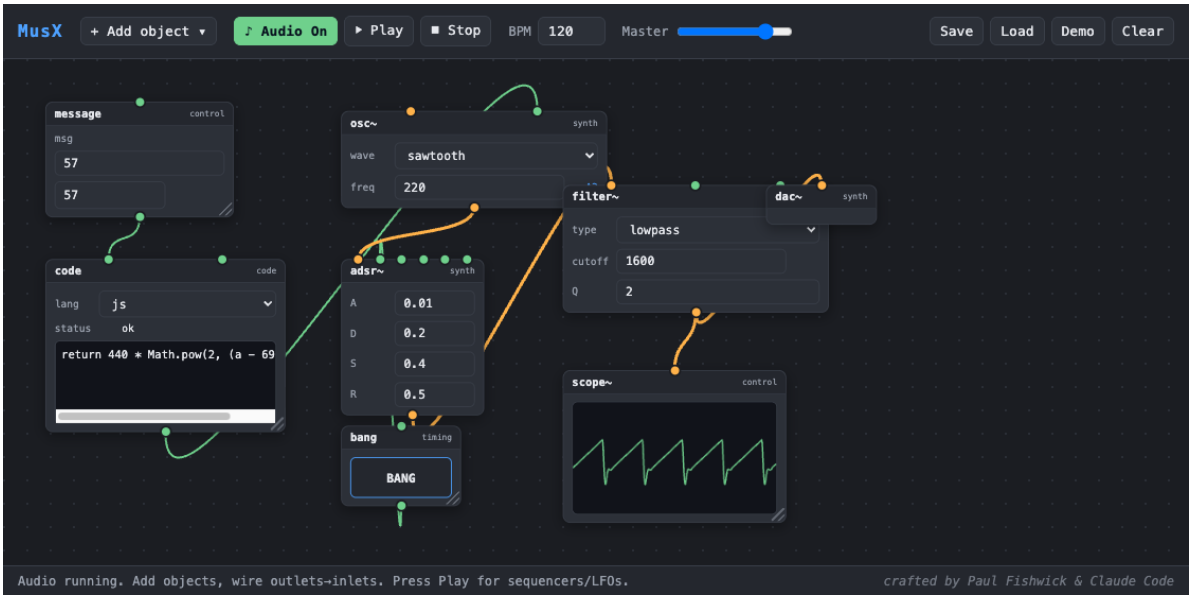

Figure 10: The "Bang + Code" patch. A `message` (MIDI 57) feeds a `code` object computing `440 * Math.pow(2,(a-69)/12)` → the oscillator's frequency; the `bang` button triggers the `adsr~`.

`pan~`. A plain equal-power stereo panner, useful for placing any mono source in the field or for widening a hand-built stack.

`spat~`. A 3D binaural panner built on the Web Audio HRTF model (`Tone.Panner3D`) [3]. The listener sits at the origin facing $-z$; three controls place the source in space: `x` left(−)/right(+), `y` down(−)/up(+), and `z` front(−)/back(+). Heard over headphones the source is localized in three dimensions, not just across the stereo pair. Because each of `x`, `y`, `z` is modulatable, wiring an `xy pad` or a slow `funcgen` into them sweeps a sound around the head in real time. (For a single moving source this HRTF panner is the natural fit; a full rotatable *soundfield*, e.g. for VR head-tracking, would call for an Ambisonics stage, a possible future addition.)

`chord`. Turns one `root` frequency into a chord. A `quality` control selects *major, minor, augmented, diminished, sus2,* or *sus4*, and a `notes` control selects how many notes sound: just the *root*, a *power* chord (root + fifth), a *triad* (1–3–5), or a *7th* (1–3–5–7). The chord tones are distributed across four frequency outlets, wrapping when there are fewer tones than outlets so that every connected voice always receives a sensible pitch — selecting *root* therefore simply doubles the root, making the chord entirely optional. The chord recomputes live as the root arrives or the quality/size change, so it can be reshaped while a note is held.

Start-Mod for samples. Stacking several copies of the *same* sample in unison causes hollow, metallic phase cancellation. The `sndfile~` object's `startmod` parameter applies a small random start-time offset (0–200 ms) on each trigger, so stacked sample voices blend into a lush ensemble instead of cancelling.

Hold to sustain. The on-screen `keyboard` plays gated notes: pressing a key opens and *sustains* the envelope, and releasing it triggers the release — so a held key sounds for as long as it is held. The

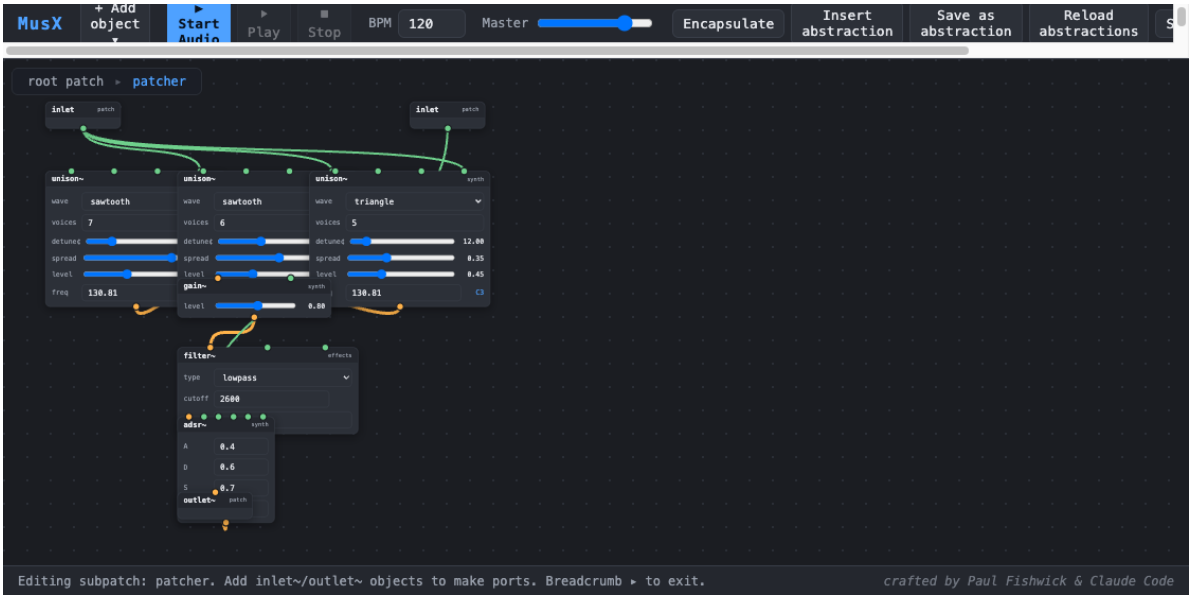

Figure 11: A `patcher` opened for editing. The breadcrumb (top-left) shows `root patch` ▷ `patcher`; inside, `inlet`/`outlet~` boundary objects define the box's ports while the inner objects do the work.

`adsr~` envelope honours these note-on/note-off messages, while the `note` object and step sequencer keep their original fixed-duration triggering.

Sampled voices (`sampler~`). The fat, "cinematic" side of instruments like Hexeract is largely *sample*-driven: real recordings (choirs, strings, brass) played back per key. The `sampler~` object provides this — it plays a loaded sample at a playback rate set by the incoming `freq` relative to a `root` pitch (varispeed), with a built-in attack/release gate so a held key sustains (looping the sample) and releases cleanly, plus the same Start-Mod jitter for stacking. To sustain smoothly, the loop's seam is equal-power crossfaded and its start (`loop→`, in milliseconds) is moved past the onset attack, so the loop sits in the steady-state part of the sample rather than re-triggering the attack each cycle. Crucially, because it is driven by `freq` exactly like an oscillator, it drops into the same `keyboard` + `chord` rig: replacing the synth voices with `sampler~` voices turns the chord patch into a *sampled* chord with no other change (see the demos below).

10 Built-in Demo Patches

Nine demos ship with MusX. Two reproduce examples from the Max/MSP product page [2]. **Custom Synth** (Figure 12) recreates the first "Sound without limits" patch: stacked saw and square oscillators through a resonant filter, distortion, and delay, driven by the step sequencer. **Layered Pad** (Figure 13) recreates the third patch, which in Max uses multichannel (MC) objects to stack fifty detuned oscillators; MusX emulates this with a stack of detuned oscillators feeding an LFO-swept filter and reverb. **Richsound Chord** (Figure 14) shows the fat-voice objects at work: a `keyboard` drives a `chord` object whose tones set three `unison~`-based voices (each a referenced *richsound* abstraction, Section 8), so a single held key sustains a wide, transposable chord (Section 9). Two further demos show the sampled path: **Sampler (playable)** plays a bundled "ah" vowel chromatically from the keyboard, and **Sampled Chord** is the identical keyboard + `chord`

rig with the three voices replaced by `sampler~` objects, giving a choir-like sampled chord. **3D Orbit** sends a bright sawtooth through `spat~` while two `funcgen` LFOs circle it around the listener (put on headphones), and **Cathedral Pad** is a playable version of the full "fat & massive" recipe: a keyboard feeds a `chord` that pitches two detuned `unison~` voices to the played root and fifth, a `math` node drops a sine an octave below for sub weight, and an `adsr~` (sustain = 1) holds the sound for as long as the key is down. The signal is tamed by a lowpass and gently saturated, then sent to a long, pre-delayed `reverb~` whose send is high-passed for a clean tail. **Cathedral (MIDI, polyphonic)** plays a real `.mid` file through a bank of eight Cathedral voices: the `midifile` object parses the file, and its built-in voice allocator hands each note to a free voice (one `freq`/`trig` outlet pair per voice, exactly like the keyboard), so the piece sounds as actual chords summed into one shared reverb. **Cathedral (MIDI, monophonic)** plays the same file through the single mono pad instead: the `midifile` object's `mode` is set to *mono*, so it follows the highest held note and keeps the gate open through overlaps (legato), turning the dense piece into one continuous, evolving line — cleaner than the polyphonic version, which can wash out where the music is busiest. Both demos also carry a *credit*: patches may include an optional `credits` field (`text` + `url`) that the viewer shows in a small box in the top-right corner. **UCI Arts — Mono MIDI Synth** is a faithful replica of Christopher Dobrian's Max Cookbook patch *Very Simple Monophonic MIDI Synthesizer*: the `midifile` object in *mono* mode with *retrig* on stands in for Max's `poly 1 1` (mono, note-stealing), its frequency drives a sawtooth `osc~` (`mtof`), and the note-on velocity scales the amplitude through an `adsr~` with *vel→amp* enabled (velocity $1 \ldots 127$ mapped to $-60 \ldots 0$ dB, then `dbtoa`). It is credited to © 2017 Christopher Dobrian (UCI Arts). The `midifile` object's `track` (isolate one part), `start` (skip seconds), `mode` (poly/mono), and `retrig` controls make all of this configurable live. The remaining demos are **XY Synth** (Figure 1), **FuncGen → Plot** (Figure 9), **Keyboard Synth**, and **Bang + Code** (Figure 10).

Figure 12: The "Custom Synth" demo (Max "Sound without limits", patch 1): a step sequencer drives saw + square oscillators through an envelope, resonant lowpass filter, distortion, and delay, with a scope on the output.

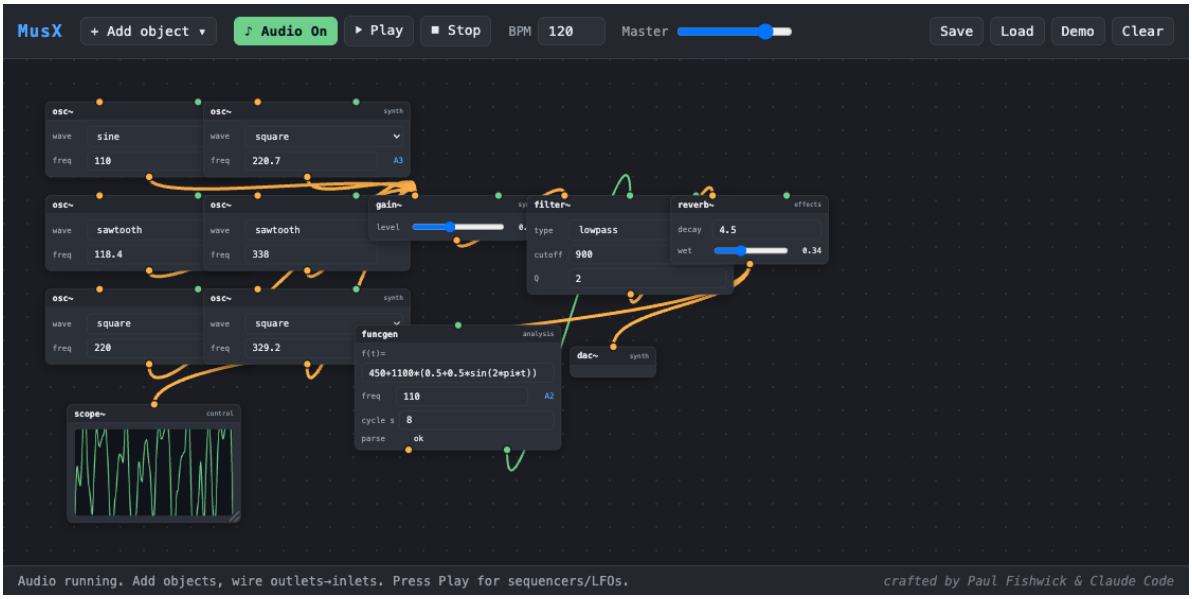

Figure 13: The "Layered Pad" demo (Max "Sound without limits", patch 3): six detuned oscillators are mixed and passed through a filter whose cutoff is swept by a slow `funcgen` LFO, then reverberated. This emulates the MC (multichannel) oscillator bank of the original.

11 Architecture

MusX separates two layers. The *graph layer* (`js/graph/`) holds a serialisable model of objects and connections and renders the boxes, ports, and cables; it knows nothing about audio. The *audio layer* (`js/audio/engine.js` plus the per-object definitions in `js/nodes/`) builds the live Tone.js [4] graph from the model. Audio cables become real Tone connections; control cables are delivered dynamically — when an object emits, the engine looks up the current control connections and calls the targets — so editing cables while running takes effect immediately. A single master gain node sits between every `dac~` and the Web Audio destination [3]; the transport's Stop ramps it to zero, which is how Stop reliably silences free-running drones.

12 Developer Reference: Adding a New Object

An object type is one entry in the node registry. A definition declares its title, category, inlets, outlets, parameter widgets, an optional `render` for custom UI, and a `create` that builds its Tone.js object(s) and returns a small runtime (`audioIn`, `audioOut`, `receive`, `setParam`, `start`, `dispose`). To make an object resizable, set `resizable: true` and, in `render`, assign `view._vizEl` (the element to measure) and `view._onResize(w, h)` (how to apply a new size). No change to the graph engine is required.

13 Testing

MusX ships with a headless test harness (`tests/auto/test.mjs`) that loads each demo and verifies both structure and sound. Because a browser cannot "hear" audio, the harness taps a meter off the master and reads the live level in decibels, confirming that a patch produces real signal rather than

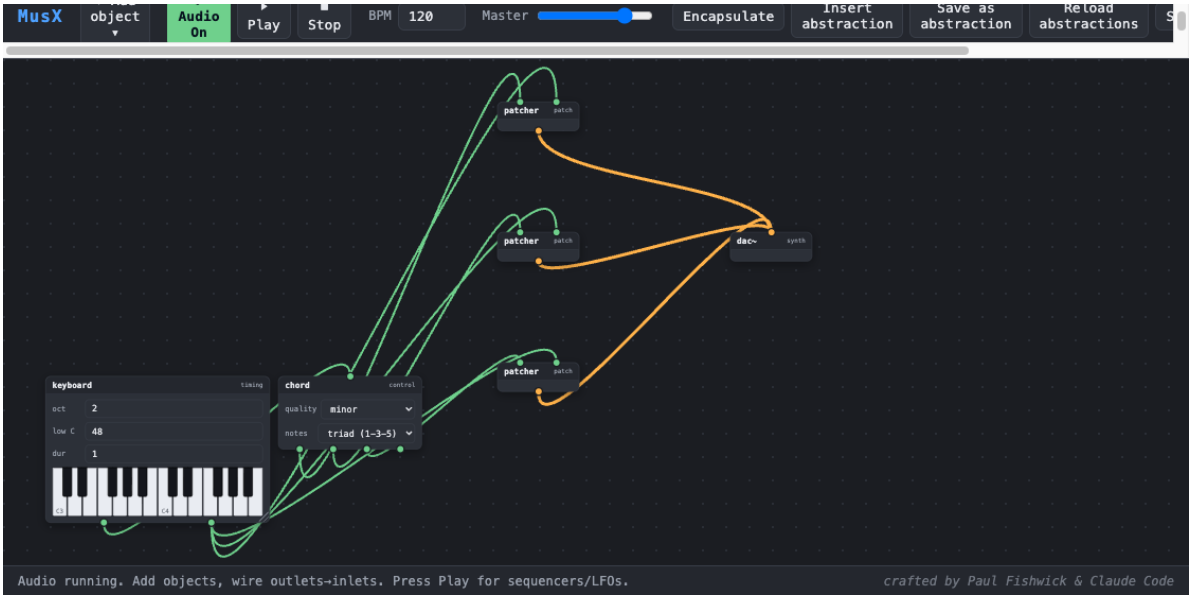

Figure 14: The "Richsound Chord" demo: a `keyboard` feeds a `chord` object, whose four frequency outlets drive three `unison~`-based voices (referenced *richsound* abstractions). The one keyboard gate holds all three envelopes, so a held key sustains a wide, transposable chord; the chord object's quality and size can be changed live.

silence, that Stop drives the output to zero, and that the keyboard and bang trigger audible notes. The manual's figures are produced separately by a Selenium script (`docs/make_figures.py`) that drives the same interface in headless Chrome.

References

[1] M. Puckette. "Pure Data: another integrated computer music environment." *Proceedings of the International Computer Music Conference*, 1996. Software: https://puredata.info.

[2] Cycling '74. *Max* (Max/MSP). https://cycling74.com/products/max.

[3] W3C. *Web Audio API*. https://www.w3.org/TR/webaudio/.

[4] Y. Mann. *Tone.js: A Web Audio framework for interactive music in the browser*. https://tonejs.github.io.

[5] The Pyodide development team. *Pyodide: Python with the scientific stack, compiled to WebAssembly*. https://pyodide.org.

[6] The Composers Desktop Project. *CDP: a suite of sound-transformation processes for* musique concrète. https://www.composersdesktop.com; source: https://github.com/ComposersDesktop/CDP8.

[7] J.-P. Higgins. *SoundThread: a node-based graphical interface for the Composers Desktop Project*. https://github.com/j-p-higgins/SoundThread.

[8] HeavyMetal. *Hexeract: a layered-oscillator software synthesizer*. Cited as the design inspiration for the fat unison/detune/spread voice architecture.